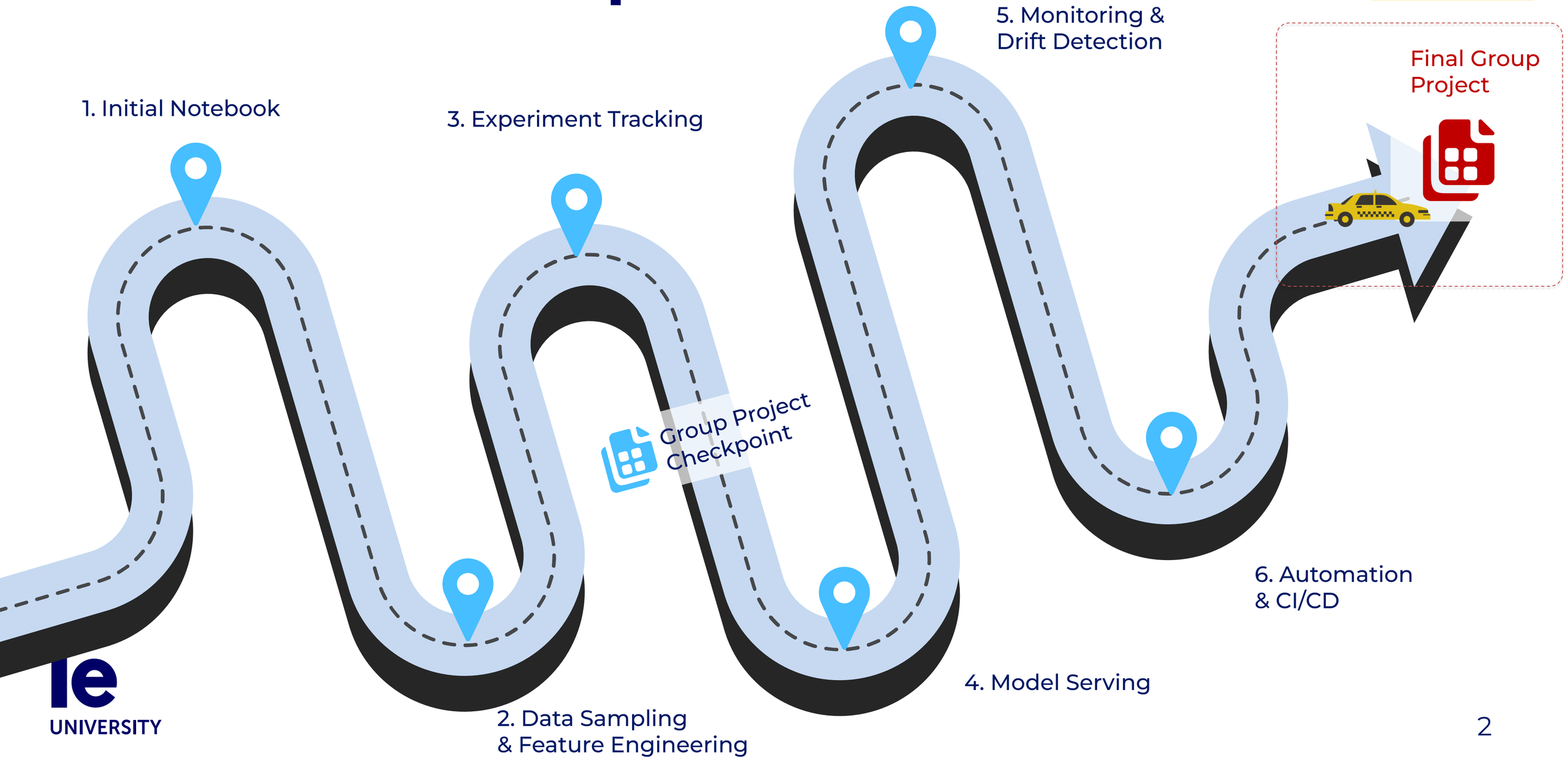
Final Group Project

Machine Learning Operations

Hands-On Roadmap



Evaluation methodology

Let's Start!

Course Start
Session 1

20%
Group Project Checkpoint
Session 8

Proposal: business problem, data, modeling, system

10%
Intermediate Test
Session 9

Initial core concepts and fundamental best practices

30%
Final Group Project
Session 13/14

Final group project (Build an ML-powered application)

30%
Final Exam
Session 15

Final closed-book exam

10%
Class Participation
On-going

Relevant contributions and participation in class

Bad sign if by the end of the course, I still don't know who you are

ie UNIVERSITY

Final Group Project

Final Group Project: Build an ML-powered application

- Each team will build and present an end-to-end ML system that goes from training → deployment → CI/CD automation, demonstrating both business impact and technical soundness.

📄 Deliverable (GitHub Repository)

Must include:

- Model training script (train.py, MLflow tracking)
- Serving API (app.py, FastAPI)
- Dockerfile (containerized service)
- CI/CD workflow (.github/workflows/ci-cd.yml)
    - Lint + test + build + deploy
- Deployment manifest (render.yaml)
    - Working online endpoint (Render.com or equivalent)
- README.md explaining setup, workflow, and usage

Ensure best practices and a proper project structure are used.

💻 Demo in Class (15 mins per team)

Part 1 – Business View (5 min)

- What problem does it solve?
- Why is the model useful or valuable?
- How could this be integrated in a real company workflow?

Part 2 – Technical Walkthrough (10 min)

- Show your training and deployment flow
- Explain how your pipeline works
- Live test: call your /predict endpoint or UI demo
- Reflect on lessons learned / main challenges

Due date: 16 March (the day before Sessions 13/14: Demo Day)

Final Group Project

Expectations

General Best Practices

1. **Modularization:** Break down code into smaller, reusable functions or classes.
2. **Configuration Management:** Use configuration files (config.yaml) or environment variables for settings.
3. **Logging and Monitoring:** Add logging (MLFlow) and possibly some basic monitoring.
4. **Error Handling:** Include comprehensive error handling and data validation.
5. **Dependency Management:** List all dependencies and versions for reproducibility.
6. **Documentation:** Include sufficient comments and documentation for maintainability.
7. **Code Quality:** Ensure the code is clean, readable, and follows PEP 8 standards.
8. **Testing:** Include unit tests to validate the functionality.
9. **Security:** Remove any sensitive or secret info (.env).